L09 — Quick Sort: Algorithm and Complexity Analysis

Unit: 1 | **Chapter:** Sorting Techniques | **BT Level:** BT4 | **CO:** CO1, CO5

Learning Objectives

- Explain the divide-and-conquer paradigm as applied to quick sort
- Implement quick sort with the Lomuto and Hoare partition schemes
- Derive the time complexity for best, worst, and average cases
- Explain the role of pivot selection and its impact on performance

---

1. Overview

Quick Sort is a **divide-and-conquer** sorting algorithm that selects a **pivot** element and partitions the array into two sub-arrays:

- Elements less than or equal to the pivot (left side)
- Elements greater than the pivot (right side)

The sub-arrays are then recursively sorted.

Why "quick"? It has small constant factors and excellent cache performance, making it the fastest sorting algorithm in practice for most inputs.

---

2. Lomuto Partition Scheme

2.1 Partition Algorithm

The pivot is always the **last element**. Elements smaller than the pivot are moved to the left.

```
def lomuto_partition(arr, low, high):
    pivot = arr[high]   # Choose last element as pivot
    i = low - 1         # Index of smaller element boundary

    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]  # Swap

    arr[i + 1], arr[high] = arr[high], arr[i + 1]  # Place pivot
    return i + 1        # Return pivot's final index
```

2.2 Quick Sort with Lomuto

```
def quick_sort(arr, low, high):
    if low < high:
        pivot_idx = lomuto_partition(arr, low, high)
        quick_sort(arr, low, pivot_idx - 1)   # Sort left of pivot
        quick_sort(arr, pivot_idx + 1, high)  # Sort right of pivot
```

---

3. Partition Trace

Array: `[10, 7, 8, 9, 1, 5]`, low = 0, high = 5, **pivot = 5**

j	arr[j]	<= pivot?	i	Array
0	10	No	-1	[10, 7, 8, 9, 1, 5]
1	7	No	-1	[10, 7, 8, 9, 1, 5]
2	8	No	-1	[10, 7, 8, 9, 1, 5]
3	9	No	-1	[10, 7, 8, 9, 1, 5]
4	1	Yes	0	[**1**, 7, 8, 9, 10, 5]

Place pivot: swap arr[1] and arr[5] → `[1, **5**, 8, 9, 10, 7]`

Pivot 5 is now at index 1. Left: [1], Right: [8, 9, 10, 7] → recurse on both.

---

4. Hoare Partition Scheme

Faster in practice (3× fewer swaps on average):

```
def hoare_partition(arr, low, high):
    pivot = arr[low + (high - low) // 2]  # Middle as pivot
    i = low - 1
    j = high + 1

    while True:
        i += 1
        while arr[i] < pivot:
            i += 1
        j -= 1
        while arr[j] > pivot:
            j -= 1
        if i >= j:
            return j
        arr[i], arr[j] = arr[j], arr[i]

def quick_sort_hoare(arr, low, high):
    if low < high:
        pivot_idx = hoare_partition(arr, low, high)
        quick_sort_hoare(arr, low, pivot_idx)
        quick_sort_hoare(arr, pivot_idx + 1, high)
```

---

5. Complexity Analysis

5.1 Recurrence Relations

Best / Average Case: Pivot splits array into two roughly equal halves: $$T(n) = 2T(n/2) + O(n)$$

By Master Theorem: **T(n) = O(n log n)**

Worst Case: Pivot is always the smallest or largest element (e.g., sorted array with last-element pivot): $$T(n) = T(n-1) + T(0) + O(n) = T(n-1) + O(n)$$

Unrolling: T(n) = O(n) + O(n-1) + ... + O(1) = **$O(n^2)$**

5.2 Summary Table

Case	Condition	Time	Space (call stack)
Best	Balanced splits at every step	O(n log n)	O(log n)
Average	Random input	O(n log n)	O(log n)
Worst	Already sorted + bad pivot	$O(n^2)$	O(n)

---

6. Pivot Selection Strategies

The choice of pivot critically affects performance.

Strategy	Description	Handles Sorted Input
Last element (Lomuto)	Simple, but $O(n^2)$ on sorted arrays	No
First element	Same issue as last element	No
Random element	Avoids worst case with high probability	Yes
Median-of-three	Picks median of first, middle, last	Yes

Randomized Quick Sort

```
import random

def randomized_partition(arr, low, high):
    rand_idx = random.randint(low, high)
    arr[rand_idx], arr[high] = arr[high], arr[rand_idx]  # Swap random
    return lomuto_partition(arr, low, high)
```

Expected time: **O(n log n)** regardless of input.

---

7. Quick Sort vs Other O(n log n) Sorts

Property	Quick Sort	Merge Sort	Heap Sort
Best case	O(n log n)	O(n log n)	O(n log n)

Average case	O(n log n)	O(n log n)	O(n log n)
Worst case	$O(n^2)$	O(n log n)	O(n log n)
Space	O(log n)	O(n)	O(1)
Stable	No	Yes	No
Cache perf.	Excellent	Good	Poor

Quick sort is preferred in practice due to its **excellent cache performance** and small constants, despite its $O(n^2)$ worst case.

---

8. Full Working Example

```
def quick_sort_full(arr, low=0, high=None):
    if high is None:
        high = len(arr) - 1
    if low < high:
        pi = lomuto_partition(arr, low, high)
        quick_sort_full(arr, low, pi - 1)
        quick_sort_full(arr, pi + 1, high)
    return arr

arr = [3, 6, 8, 10, 1, 2, 1]
print(quick_sort_full(arr))
# Output: [1, 1, 2, 3, 6, 8, 10]
```

---

9. Tail Call Optimization

To reduce stack depth from O(n) to O(log n) in the worst case, always recurse on the smaller partition first:

```
def quick_sort_optimized(arr, low, high):
    while low < high:
        pi = lomuto_partition(arr, low, high)
        if pi - low < high - pi:     # Left partition is smaller
            quick_sort_optimized(arr, low, pi - 1)
            low = pi + 1             # Tail call on right
        else:
            quick_sort_optimized(arr, pi + 1, high)
            high = pi - 1            # Tail call on left
```

Summary

- Quick sort uses divide-and-conquer: partition around a pivot, then sort sub-arrays recursively.
- Average and best case: **O(n log n)**; worst case: **$O(n^2)$** (avoided with random pivot).
- **In-place** (O(log n) stack space), but **not stable**.
- Preferred for general-purpose sorting due to cache efficiency and small constants.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 7 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 4.6 (Springer, 2020)
- Laaksonen, *Guide to Competitive Programming*, Ch. 3.1 (Springer, 2024)